1.(i) BASIC LINUX ENVIRONMENT AND ITS EDITORS LIKE VI,VIM & EMACS

Basic Linux Commands:

ls: List files and directories.

cd: Change directory.

pwd: Print current working directory.

cp: Copy files or directories.

mv: Move or rename files or directories.

rm: Remove files or directories.

mkdir: Create a new directory.

cat: Concatenate and display file content.

grep: Search for a pattern in files.

chmod: Change file permissions.

ps: Display information about processes.

top: Display and update sorted information about processes.

Vi / Vim Editor Commands:

To open a file: vim filename

Switch to insert mode (for editing): Press i

Save changes and exit: Press Esc, then type :wq and press Enter

Exit without saving changes: Press Esc, then type :q! and press Enter

Emacs Editor Commands:

To open a file: emacs filename

Save changes: Ctrl + x, Ctrl + s

Save changes and exit: Ctrl + x, Ctrl + c

Exit without saving changes: Ctrl + x, Ctrl + c (followed by n for "no").

1.(ii) EXPOSURE TO TURBO C, GCC

PROGRAM TEXT:

Turbo C:

Target Platform:

Turbo C was primarily designed for the DOS operating system and the Windows 3.x environment.

Usage in Education:

It gained popularity in the 1990s and was commonly used as a teaching tool for introductory C programming courses.

Integrated Development Environment (IDE):

Turbo C provides an integrated environment with a text editor, compiler, and linker, simplifying the development process.

Graphics and DOS-based Applications:

It was often used for developing simple graphics applications and DOS-based programs.

GCC (GNU Compiler Collection):

Cross-Platform Support:

GCC is a versatile compiler that supports multiple platforms, including Unix/Linux, macOS, and Windows.

Open Source:

GCC is open-source software, allowing developers to inspect, modify, and distribute the source code.

Industry Standard:

It has become an industry standard for C and C++ compilation on various platforms.

Command-Line Interface:

Unlike Turbo C, GCC is typically used through the command line, providing flexibility and configurability.

Optimization:

GCC offers advanced optimization options, making it suitable for high-performance applications.

Common Uses for Both Compilers:

C Programming:

Both Turbo C and GCC are used for compiling C programs.

Learning C Language:

Turbo C is often used in educational settings for teaching C programming basics, while GCC is suitable for a broader range of applications.

Summary:

Turbo C is historically associated with DOS and Windows 3.x environments, commonly used in educational settings, while GCC is a versatile, open-source compiler widely used across different platforms for various applications.

1.(iii) WRITING SIMPLE PROGRAM USING PRINTF( ),SCANF( )

SOURCE CODE:

```
#include<stdio.h>
int main()
{
int number;
printf("Enter the number");
scanf("%d",&number);
printf("Cube of Number is %d", number*number*number);
return 0;
}
```

2.(i) SUM AND AVERAGE OF 3 NUMBERS

SOURCE CODE:

```
// C Program to Find Sum and Average of 3 Numbers
#include <stdio.h>
int main(){
int a, b, c, sum;
float avg;
// Asking for input
printf("Enter 3 numbers: \n");
scanf("%d %d %d", &a, &b, &c);
// Calculating sum
sum = a + b + c;
// Calculating average of 3 numbers
avg = (float)(sum) / 3;
// Displaying output
printf("Sum = %d \n", sum);
printf("Average = %.2f", avg);
return 0;
}
```

2.(ii) CONVERSION OF FAHRENHEIT TO CELSIUS AND CELSIUS TO FAHRENHEIT

SOURCE CODE:

```
#include <stdio.h>
int main() {
// Fahrenheit to Celsius conversion
float fahrenheit, celsius;
printf("Enter temperature in Fahrenheit: ");
scanf("%f", &fahrenheit);
celsius = (fahrenheit - 32) * 5/9;
printf("%.2f Fahrenheit is equal to %.2f Celsius\n", fahrenheit, celsius);
// Celsius to Fahrenheit conversion
printf("\nEnter temperature in Celsius: ");
scanf("%f", &celsius);
fahrenheit = celsius * 9/5 + 32;
printf("%.2f Celsius is equal to %.2f Fahrenheit\n", celsius, fahrenheit);
return 0;
}
```

2.(iii) SIMPLE INTEREST CALCULATION

SOURCE CODE:

```
#include <stdio.h>
int main() {
float principal, rate, time, simple_interest;
// Input principal amount, rate of interest, and time
printf("Enter principal amount: ");
scanf("%f", &principal);
printf("Enter rate of interest: ");
scanf("%f", &rate);
printf("Enter time in years: ");
scanf("%f", &time);
// Calculate simple interest
simple_interest = (principal * rate * time) / 100;
// Display the result
printf("Simple Interest = %.2f\n", simple_interest);
return 0;
}
```

3.(i) FINDING THE SQUARE ROOT OF A GIVEN NUMBER

SOURCE CODE:

```
#include <stdio.h>
#include <math.h>
int main() {
double number, squareRoot;
// Input the number
printf("Enter a number: ");
scanf("%lf", &number);
// Calculate the square root
squareRoot = sqrt(number);
// Display the result
printf("Square root of %.2f = %.2f\n", number, squareRoot);
return 0;
}
```

3.(ii) FINDING COMPOUND INTEREST

SOURCE CODE:

```
#include <stdio.h>
#include <math.h>
int main()
{
 float principle, rate, time, CI;
 printf("Enter principle (amount): ");
 scanf("%f", &principle);
 printf("Enter time: ");
 scanf("%f", &time);
 printf("Enter rate: ");
 scanf("%f", &rate);
 CI = principle* (pow((1 + rate / 100), time));
 printf("Compound Interest = %f", CI);
 return 0;
}
```

3.(iii) AREA OF A TRIANGLE USING HERON'S FORMULAE

SOURCE CODE:

```
#include<stdio.h>
#include<math.h>
int main()
{
float a, b, c, s, Area;
printf("\nPlease Enter three sides of triangle\n");
scanf("%f%f%f",&a,&b,&c);
s = (a+b+c)/2;
Area = sqrt(s*(s-a)*(s-b)*(s-c));
printf("\n Area of triangle = %.2f\n",Area);
return 0;
}
```

3.(iv) DISTANCE TRAVELLED BY AN OBJECT

SOURCE CODE:

```
#include <stdio.h>
int main()
{
double speed; // Uniform speed in meters per second
double time; // Time in seconds
double distance; // Distance traveled in meters
// Input the speed and time from the user
printf("Enter the speed (m/s): ");
scanf("%lf", &speed);
printf("Enter the time (s): ");
scanf("%lf", &time);
// Calculate the distance using the formula: Distance = Speed × Time
distance = speed * time;
// Display the result
printf("Distance traveled: %.2f meters\n", distance);
return 0;
}
```

4.(i) EVALUATE THE FOLLOWING EXPRESSION

A. A+B*C+(D*E) + F*G

SOURCE CODE:

```
#include <stdio.h>
int main()
{
// Define variables A, B, C, D, E, F, and G with some sample values
int A = 3, B = 2, C = 4, D = 5, E = 6, F = 7, G = 8, result;
// Evaluate the expression
result = A + B * C + (D * E) + F * G;
// Print the result
printf("Result of the expression A + B * C + (D * E) + F * G is: %d\n", result);
return 0;
}
```

B. A/B*C-B+A*D/3

SOURCE CODE:

```
#include <stdio.h>
int main()
{
 // Define variables A, B, C, and D with some sample values
 int A = 5, B = 2, C = 3, D = 4;
 // Evaluate the new expression
 int result = A / B * C - B + A * D / 3;
 // Print the result
 printf("Result of the new expression A / B * C - B + A * D / 3 is: %d\n", result);
 return 0;
}
```

C. A++ + B-- - A

SOURCE CODE:

```
#include <stdio.h>
int main()
{
int A = 5;
int B = 2;
int result = A++ + B-- - A;
printf("Result: %d\n", result);
printf("A: %d\n", A);
printf("B: %d\n", B);
return 0;
}
```

D. J= (i++) + (++i)

SOURCE CODE:

```
#include <stdio.h>
int main()
{
int i = 5;
int J = (i++) + (++i);
printf("i: %d\n", i);
printf("J: %d\n", J);
return 0;
}
```

4.(ii) FIND THE MAXIMUM OF THREE NUMBERS USING CONDITIONAL OPERATOR

SOURCE CODE:

```
#include <stdio.h>

int main()

{

int num1, num2, num3, max;

printf("Enter three numbers: ");

scanf("%d %d %d", &num1, &num2, &num3);

// Using conditional (ternary) operator to find the maximum

max = (num1 > num2) ? ((num1 > num3) ? num1 : num3) : ((num2 > num3) ? num2 :
num3);

printf("Maximum of the three numbers is: %d\n", max);

return 0;

}
```

4.(iii) TAKE MARKS OF 5 SUBJECTS IN INTEGERS, AND FIND THE TOTAL, AVERAGE IN FLOAT

SOURCE CODE:

```
#include<stdio.h>
int main()
{
int chem,phy,eng,maths,bio;
float total,average,percentage;
//taking input
printf("Enter the marks of five subjects: \n");
scanf("%d%d%d%d%d", &chem, &phy, &eng, &maths, &bio);
//sum total
total= chem + phy + eng + maths + bio;
average = total / 5;
//printing the values
printf("Total marks = %.2f \n", total);
printf("Average marks = %.2f \n", average);
return 0;
}
```

5.(i) WRITE A C PROGRAM TO FIND THE MAX AND MIN OF FOUR NUMBERS USING IF-ELSE

SOURCE CODE:

```
#include <stdio.h>
int main()
{
// Declare variables to store four numbers
int num1, num2, num3, num4;
// Read four numbers from the user
printf("Enter four numbers: ");
scanf("%d %d %d %d", &num1, &num2, &num3, &num4);
// Assume num1 as the initial values for max and min
int max = num1;
int min = num1;
// Compare num2 with current max and min
if (num2 > max)
{
max = num2;
}
else if (num2 < min)
{
min = num2;
}
// Compare num3 with current max and min
if (num3 > max)
{
```

```
max = num3;
}
else if (num3 < min)
{
min = num3;
}
// Compare num4 with current max and min
if (num4 > max)
{
max = num4;
}
else if (num4 < min)
{
min = num4;
}
// Print the maximum and minimum values
printf("Maximum: %d\n", max);
printf("Minimum: %d\n", min);
return 0;
}
```

5.(ii) WRITE A C PROGRAM TO GENERATE ELECTRICITY BILL

SOURCE CODE:

```
#include <stdio.h>
int main(){
int units;
float amt, unitcharg, total;
printf(" Enter no of units consumed : ");
scanf("%d", &units);
if (units < 50){
amt = units * 3.50;
unitcharg = 25;}
else if (units <= 100){
amt = 130 + ((units - 50 ) * 4.25);
unitcharg = 35;}
else if (units <= 200){
amt = 130 + 162.50 + ((units - 100 ) * 5.26);
unitcharg = 45;}
else{
amt = 130 + 162.50 + 526 + ((units - 200 ) * 7.75);
unitcharg = 55;}
total= amt+ unitcharg;
printf("electricity bill = %.2f", total);
return 0;
}
```

5.(iii) FIND THE ROOTS OF THE QUADRATIC EQUATION

SOURCE CODE:

```
#include <stdio.h>
#include <math.h>
int main()
{
// Declare variables to store coefficients
float a, b, c;
// Prompt the user to enter coefficients
printf("Enter coefficients (a, b, c) of the quadratic equation (ax^2 + bx + c): ");
scanf("%f %f %f", &a, &b, &c);
// Calculate the discriminant
float discriminant = b * b - 4 * a * c;
// Check the nature of roots based on the discriminant
if (discriminant > 0)
{
float root1 = (-b + sqrt(discriminant)) / (2 * a);
float root2 = (-b - sqrt(discriminant)) / (2 * a);
printf("Roots are real and different.\n");
printf("Root 1 = %.2f\nRoot 2 = %.2f\n", root1, root2);
}
else if (discriminant == 0)
{
float root = -b / (2 * a);
printf("Roots are real and the same.\n");
printf("Root = %.2f\n", root);
```

```
}
else
{
float realPart = -b / (2 * a);
float imaginaryPart = sqrt(-discriminant) / (2 * a);
printf("Roots are complex and different.\n");
printf("Root 1 = %.2f + %.2fi\n", realPart, imaginaryPart);
printf("Root 2 = %.2f - %.2fi\n", realPart, imaginaryPart);
}
return 0;
}
```

5.(iv) WRITE A C PROGRAM TO SIMULATE A CALCULATOR USING SWITCH CASE

SOURCE CODE:

```
#include <stdio.h>

int main()

{

// Declare variables

float num1, num2, result;

char operator;

// Prompt the user to enter numbers and operator

printf("Enter first number: ");

scanf("%f", &num1);

printf("Enter an operator (+, -, *, /): ");

scanf(" %c", &operator); // Note the space before %c to consume the newline
character.

printf("Enter second number: ");

scanf("%f", &num2);

// Perform arithmetic operation based on the operator

switch (operator)

{

case '+':

result = num1 + num2;

break;

case '-':

result = num1 - num2;

break;

case '*':
```

```
result = num1 * num2;
break;
case '/':
// Check if the denominator is not zero
if (num2 != 0)
{
result = num1 / num2;
}
else
{
printf("Error: Division by zero is undefined.\n");
return 1; // Exit with an error code
}
break;
default:
printf("Error: Invalid operator.\n");
return 1; // Exit with an error code
}
// Print the result
printf("Result: %.2f\n", result);
return 0;
}
```

5.(v) WRITE A C PROGRAM TO FIND THE GIVEN YEAR IS LEAP YEAR OR NOT

SOURCE CODE:

```
#include <stdio.h>
int main(){
// Declare a variable to store the year
int year;
// Prompt the user to enter a year
printf("Enter a year: ");
scanf("%d", &year);
// Check the leap year conditions
if (year % 400 == 0){
// Leap year if perfectly divisible by 400
printf("%d is a leap year.", year);}
else if (year % 100 == 0){
// Not a leap year if divisible by 100 but not by 400
printf("%d is not a leap year.", year);}
else if (year % 4 == 0){
// Leap year if not divisible by 100 but divisible by 4
printf("%d is a leap year.", year);}
else{
// All other years are not leap years
printf("%d is not a leap year.", year);}
return 0;
}
```

6.(i) FIND THE FACTORIAL OF GIVEN NUMBER USING ANY LOOP

SOURCE CODE:

```
#include<stdio.h>
int main()
{
int i,fact=1,number;
printf("Enter a number: ");
scanf("%d",&number);
for(i=1;i<=number;i++){
fact=fact*i;
}
printf("Factorial of %d is: %d",number,fact);
return 0;
}
```

6.(ii) FIND THE GIVEN NUMBER IS A PRIME OR NOT

SOURCE CODE:

```
#include<stdio.h>
int main()
{
int n,i,m=0,flag=0;
printf("Enter the number to check prime:");
scanf("%d",&n);
m=n/2;
for(i=2;i<=m;i++)
{
if(n%i==0)
{
printf("Number is not prime");
flag=1;
break;
}
}
if(flag==0)
printf("Number is prime");
return 0;
}
```

6.(iii) COMPUTE SINE AND COS SERIES

SOURCE CODE:

SOURCE CODE FOR SIN SERIES:

```
#include<stdio.h>
#include<conio.h>
int main()
{
int i,n;
float x, sum, t;
printf(" Enter the value for x : ");
scanf("%f",&x);
printf(" Enter the value for n : ");
scanf("%d",&n);
x=x*3.14159/180;
t=x;
sum=x;
/* Loop to calculate the value of Sine */
for(i=1;i<=n;i++)
{
t=(t*(-1)x*x)/(2*i*(2*i+1));
sum=sum+t;
}
printf(" The value of Sin(%f) = %.4f",x,sum);
return 0;
}
```

SOURCE CODE FOR COS SERIES:

```
#include<stdio.h>
#include<conio.h>
int main()
{
int i, n;
float x, sum=1, t=1;
printf(" Enter the value for x : ");
scanf("%f",&x);
printf(" Enter the value for n : ");
scanf("%d",&n);
x=x*3.14159/180;
/* Loop to calculate the value of Cosine */
for(i=1;i<=n;i++)
{
t=t*(-1)*x*x/(2*i*(2*i-1));
sum=sum+t;
}
printf(" The value of Cos(%f) is : %.4f", x, sum);
return 0;
}
```

6.(iv) CHECKING A NUMBER PALINDROME\

SOURCE CODE:

```
#include<stdio.h>
int main()
{
int n,r,sum=0,temp;
printf("enter the number=");
scanf("%d",&n);
temp=n;
while(n>0)
{
r=n%10;
sum=(sum*10)+r;
n=n/10;
}
if(temp==sum)
printf("palindrome number ");
else
printf("not palindrome");
return 0;
}
```

6.(v) CONSTRUCT A PYRAMID OF NUMBER

SOURCE CODE:

```
#include <stdio.h>
int main(){
int i, space, rows, k = 0, count = 0, count1 = 0;
printf("Enter the number of rows: ");
scanf("%d", &rows);
for (i = 1; i <= rows; ++i){
for (space = 1; space <= rows - i; ++space){
printf(" ");
++count;}
while (k != 2 * i - 1) {
if (count <= rows - 1) {
printf("%d ", i + k);
++count;}
else{
++count1;
printf("%d ", (i + k - 2 * count1));}
++k;}
count1 = count = k = 0;
printf("\n");
}
return 0;
}
```

7.(i) FIND THE MIN AND MAX OF 1-D INTEGER ARRAY

SOURCE CODE:

```
#include <stdio.h>
#include <conio.h>
int main(){
int a[1000],i,n,min,max;
printf("Enter size of the array : ");
scanf("%d",&n);
printf("Enter elements in array : ");
for(i=0; i<n; i++)
{
scanf("%d",&a[i]);
}
min=max=a[0];
for(i=1; i<n; i++)
{
if(min>a[i])
min=a[i];
if(max<a[i])
max=a[i];
}
printf("minimum of array is : %d",min);
printf("\nmaximum of array is : %d",max);
return 0;
}
```

7.(ii) PERFORM LINEAR SEARCH ON 1-D ARRAY

SOURCE CODE:

```
#include<stdio.h>
int main()
{
int a[20],i,x,n;
printf("How many elements?");
scanf("%d",&n);
printf("Enter array elements:\n");
for(i=0;i<n;++i)
scanf("%d",&a[i]);
printf("\nEnter element to search:");
scanf("%d",&x);
for(i=0;i<n;++i)
if(a[i]==x)
break;
if(i<n)
printf("Element found at index %d",i);
else
printf("Element not found");
return 0;
}
```

7.(iii) THE REVERSE OF 1D INTEGER ARRAY

SOURCE CODE:

```
#include <stdio.h>
#define N 1000
int main()
{
int arr[N];
int n;
// Inputting the size of the array
printf("Enter the size of the array: ");
scanf("%d", &n);
// Inputting the array
printf("Enter an array: ");
for (int i = 0; i< n; i++){
scanf("%d", &arr[i]);
}
// Printing the reverse of the array
printf("Reversed array: ");
for (int i = n-1; i>=0; i--){
printf("%d ", arr[i]);
}
return 0;
}
```

7.(iv) FIND 2’S XOMPLEMENT OF THE GIVEN BINARY NUMBER

SOURCE CODE:

```
#include <stdio.h>
int main()
{
int n; // variable declaration
printf("Enter the number of bits do you want to enter :");
scanf("%d",&n);
char binary[n+1]; // binary array declaration;
char onescomplement[n+1]; // onescomplement array declaration
char twoscomplement[n+1]; // twoscomplement array declaration
int carry=1; // variable initialization
printf("\nEnter the binary number : ");
scanf("%s", binary);
printf("%s", binary);
printf("\nThe ones complement of the binary number is :");
// Finding onescomplement in C
for(int i=0;i<n;i++)
{
if(binary[i]=='0')
onescomplement[i]='1';
else if(binary[i]=='1')
onescomplement[i]='0';
}
onescomplement[n]='\0';
```

```
printf("%s",onescomplement);
printf("\nThe twos complement of a binary number is : ");
// Finding twoscomplement in C
for(int i=n-1; i>=0; i--)
{
if(onescomplement[i] == '1' && carry == 1)
{
twoscomplement[i] = '0';
}
else if(onescomplement[i] == '0' && carry == 1)
{
twoscomplement[i] = '1';
carry = 0;
}
else
{
twoscomplement[i] = onescomplement[i];
}
}
twoscomplement[n]='\0';
printf("%s",twoscomplement);
return 0;
}
```

7.(v) ELIMINATE DUPLICATE ELEMENTS IN AN ARRAY

SOURCE CODE:

```
#include <stdio.h>
#include <conio.h>
int main ()
{
// declare local variables
int arr[20], i, j, k, size;
printf (" Define the number of elements in an array: ");
scanf (" %d", &size);
printf (" \n Enter %d elements of an array: \n ", size);
// use for loop to enter the elements one by one in an array
for ( i = 0; i < size; i++)
{
scanf (" %d", &arr[i]);
}
// use nested for loop to find the duplicate elements in array
for ( i = 0; i < size; i ++)
{
for ( j = i + 1; j < size; j++)
{
// use if statement to check duplicate element
if ( arr[i] == arr[j])
{
// delete the current position of the duplicate element
```

```
for ( k = j; k < size - 1; k++)
{
arr[k] = arr [k + 1];
}
// decrease the size of array after removing duplicate element
size--;
// if the position of the elements is changes, don't increase the index j
j--;
}
}
}
/* display an array after deletion or removing of the duplicate elements */
printf (" \n Array elements after deletion of the duplicate elements: ");
// for loop to print the array
for ( i = 0; i < size; i++)
{
printf (" %d \t", arr[i]);
}
return 0;
}
```

8.(i) ADDITION OF TWO MATRICES

SOURCE CODE:

```
#include <stdio.h>
int main()
{
int m, n, i, j;
printf("Enter the number of rows and columns of the matrices: ");
scanf("%d%d", &m, &n);
int a[m][n], b[m][n], c[m][n];
printf("Enter the elements of matrix A: \n");
for (i = 0; i < m; i++)
{
for (j = 0; j < n; j++)
{
scanf("%d", &a[i][j]);
}
}
printf("Enter the elements of matrix B: \n");
for (i = 0; i < m; i++)
{
for (j = 0; j < n; j++)
{
scanf("%d", &b[i][j]);
}
}
```

```
// add the matrices
for (i = 0; i < m; i++)
{
for (j = 0; j < n; j++)
{
c[i][j] = a[i][j] + b[i][j];
}
}
// print the result
printf("The sum of the two matrices is: \n");
for (i = 0; i < m; i++)
{
for (j = 0; j < n; j++)
{
printf("%d ", c[i][j]);
}
printf("\n");
}
return 0;
}
```

8.(ii) MULTIPLICATION TWO MATRICES

SOURCE CODE:

```
#include<stdio.h>
int main(){
int r1,r2,c1,c2;
printf("Enter number of rows for First Matrix:\n");
scanf("%d",&r1);
printf("Enter number of columns for First Matrix:\n");
scanf("%d",&c1);
printf("Enter number of rows for Second Matrix:\n");
scanf("%d",&r2);
printf("Enter number of columns for Second Matrix:\n");
scanf("%d",&c2);
if(c1!=r2){
printf("Matrices Can't be multiplied together");
}
else{
int m1[r1][c1],m2[r2][c2];
printf("Enter first matrix elements \n");
for(int i=0;i<r1;i++){
for(int j=0;j<c1;j++){
scanf("%d",&m1[i][j]);
}
}
printf("Enter Second matrix elements\n");
```

```
for(int i=0;i<r2;i++){
for(int j=0;j<c2;j++){
scanf("%d",&m2[i][j]);
}
}
int mul[r1][c2];
for(int i=0;i<r1;i++){
for(int j=0;j<c2;j++){
mul[i][j]=0;
// Multiplying i’th row with j’th column
for(int k=0;k<c1;k++){
mul[i][j]+=m1[i][k]*m2[k][j];
}
}
}
printf("Multiplied matrix\n");
for(int i=0;i<r1;i++){
for(int j=0;j<c2;j++){
printf("%d\t",mul[i][j]);
}
printf("\n");
}
}
return 0;
}
```

8.(iii) SORT ARRAY ELEMENTS USING BUBBLE SORT

SOURCE CODE:

```
#include <stdio.h>
void swap(int *a, int *b)
{
int temp = *a;
*a = *b;
*b = temp;
}
void bubbleSort(int arr[], int n)
{
for (int i = 0; i < n - 1; i++)
{
for (int j = 0; j < n - i - 1; j++)
{
// Compare adjacent elements and swap if they are in the wrong order
if (arr[j] > arr[j + 1])
{
swap(&arr[j], &arr[j + 1]);
}
}
}
}
void printArray(int arr[], int size)
{
```

```
for (int i = 0; i < size; i++)
{
printf("%d ", arr[i]);
}
printf("\n");
}
int main()
{
int n;
// Get the size of the array from the user
printf("Enter the number of elements: ");
scanf("%d", &n);
int arr[n];
// Get array elements from the user
printf("Enter %d elements:\n", n);
for (int i = 0; i < n; i++)
{
scanf("%d", &arr[i]);
}
printf("Original array: ");
printArray(arr, n);
// Perform bubble sort
bubbleSort(arr, n);
printf("Sorted array: ");
printArray(arr, n);
return 0;
}
```

8.(iv) CONCATENATE TWO STRINGS WITHOUT BUILT-IN FUNCTIONS

SOURCE CODE:

```
#include <stdio.h>
void concatenateStrings(char *str1, char *str2)
{
while (*str1 != '\0')
{
str1++;
}
while (*str2 != '\0') {
*str1 = *str2;
str1++;
str2++;
}
*str1 = '\0';
}
// Function to calculate the length of a string without using strlen
size_t calculateLength(const char *str)
{
size_t length = 0;
while (*str != '\0')
{
length++;
str++;
}
```

```
return length;
}
int main(){
char firstString[50];
char secondString[50];
printf("Enter the first string: ");
fgets(firstString, sizeof(firstString), stdin);
printf("Enter the second string: ");
fgets(secondString, sizeof(secondString), stdin);
// Remove newline characters from input strings
size_t len1 = calculateLength(firstString);
if (len1 > 0 && firstString[len1 - 1] == '\n'){
firstString[len1 - 1] = '\0';
}
size_t len2 = calculateLength(secondString);
if (len2 > 0 && secondString[len2 - 1] == '\n'){
secondString[len2 - 1] = '\0';
}
printf("\nBefore concatenation:\n");
printf("First string: %s\n", firstString);
printf("Second string: %s\n", secondString);
concatenateStrings(firstString, secondString);
printf("\nAfter concatenation:\n");
printf("Concatenated string: %s\n", firstString);
return 0;
}
```

8.(v) REVERSE A STRING USING BUILT-IN AND WITHOUT BUILT-IN STRING FUNCTIONS

SOURCE CODE USING BUILD-IN FUNCTION:

```
#include <stdio.h>
#include <string.h>
void reverseString(char *str)
{
int length = strlen(str);
int start = 0;
int end = length - 1;
while (start < end)
{
char temp = str[start];
str[start] = str[end];
str[end] = temp;
start++;
end--;
}
}
int main()
{
char inputString[50];
printf("Enter a string: ");
fgets(inputString, sizeof(inputString), stdin);
// Remove the newline character if present
if (inputString[strlen(inputString) - 1] == '\n')
```

```
{
inputString[strlen(inputString) - 1] = '\0';
}
// Reverse the string
reverseString(inputString);
printf("Reversed string: %s\n", inputString);
return 0;
}
```

SOURCE CODE WITHOUT USING BUILD-IN FUNCTION:

```
#include <stdio.h>
void reverseString(char *str)
{
// Find the length of the string
int length = 0;
while (str[length] != '\0')
{
length++;
}
// Adjust length to exclude the null terminator
length--;
// Swap characters from the beginning with characters from the end
int start = 0;
int end = length;
while (start < end)
{
// Swap characters
char temp = str[start];
str[start] = str[end];
str[end] = temp;
// Move indices towards the center
start++;
end--;
}
}
int main() {
```

```
char inputString[50];
printf("Enter a string: ");
fgets(inputString, sizeof(inputString), stdin);
// Remove the newline character if present
int len = 0;
while (inputString[len] != '\0' && inputString[len] != '\n')
{
len++;
}
inputString[len] = '\0';
// Reverse the string
reverseString(inputString);
printf("Reversed string: %s\n", inputString);
return 0;
}
```

9.(i) WRITE A C FUNCTION TO CALCULATE NCR VALUE

SOURCE CODE:

```
#include <stdio.h>
int fact(int z);
void main(){
int n, r, nCr;
printf("Enter the value of n and r?");
scanf("%d %d",&n,&r);
nCr = fact(n) / (fact(r) * fact(n - r));
printf("\nnCr = %d", nCr);
}
int fact(int z){
int f = 1, i;
if (z == 0){
return(f);
}
else
{
for (i = 1; i <= z; i++){
f = f * i;
}
}
return(f);
}
```

9.(ii) WRITE A C FUNCTION TO FIND THE LENGTH OF A STRING

SOURCE CODE:

```
#include <stdio.h>
#include <string.h>
int main()
{
char Str[1000];
int i;
printf("Enter the String: ");
scanf("%s", Str);
printf("Length of Str is %ld", strlen(Str));
return 0;
}
```

9.(iii) WRITE A C FUNCTION TO TRANSPOSE OF A MATRIX

SOURCE CODE:

```
#include <stdio.h>
// Function to calculate the transpose of a matrix
void transposeMatrix(int rows, int cols, int matrix[rows][cols], int result[cols][rows])
{
for (int i = 0; i < rows; ++i)
{
for (int j = 0; j < cols; ++j)
{
result[j][i] = matrix[i][j];
}
}
}
// Function to display a matrix
void displayMatrix(int rows, int cols, int matrix[rows][cols])
{
for (int i = 0; i < rows; ++i)
{
for (int j = 0; j < cols; ++j)
{
printf("%d\t", matrix[i][j]);
}
printf("\n");
}
```

```
}
int main(){
int rows, cols;
// Input the dimensions of the matrix
printf("Enter the number of rows in the matrix: ");
scanf("%d", &rows);
printf("Enter the number of columns in the matrix: ");
scanf("%d", &cols);
int matrix[rows][cols];
// Input the matrix elements
printf("Enter the elements of the matrix:\n");
for (int i = 0; i < rows; ++i){
for (int j = 0; j < cols; ++j){
printf("Enter element at position (%d, %d): ", i + 1, j + 1);
scanf("%d", &matrix[i][j]);
}
}
// Calculate and display the transpose of the matrix
int result[cols][rows];
transposeMatrix(rows, cols, matrix, result);
printf("\nOriginal Matrix:\n");
displayMatrix(rows, cols, matrix);
printf("\nTranspose Matrix:\n");
displayMatrix(cols, rows, result);
return 0;
}
```

9.(iv) WRITE A C FUNCTION TO DEMONSTRATE NUMERICAL INTEGRATION OF DIFFERENTIAL EQUATION USING EULER'S METHOD

SOURCE CODE:

```
#include <stdio.h>
// Function representing the differential equation dy/dx = f(x, y)
double differentialEquation(double x, double y)
{
return 2 * x - y;
}
// Function to perform numerical integration using Euler's method
void eulerMethod(double x0, double y0, double h, double xn)
{
double x = x0;
double y = y0;
printf("Euler's Method:\n");
printf("x\t\t y\n");
printf("%.4f\t%.4f\n", x, y);
while (x < xn) {
y = y + h * differentialEquation(x, y);
x = x + h;
printf("%.4f\t%.4f\n", x, y);
}
}
int main() {
double x0, y0, h, xn;
```

```
// Input initial values, step size, and final x value
printf("Enter initial value x0: ");
scanf("%lf", &x0);
printf("Enter initial value y0: ");
scanf("%lf", &y0);
printf("Enter step size h: ");
scanf("%lf", &h);
printf("Enter final x value xn: ");
scanf("%lf", &xn);
// Perform numerical integration using Euler's method
eulerMethod(x0, y0, h, xn);
return 0;
```

10.(i) WRITE A RECURSIVE FUNCTION TO GENERATE FIBONACCI SERIES

SOURCE CODE:

```
#include <stdio.h>
// Recursive function to generate Fibonacci series
int fibonacci(int n)
{
if (n <= 1)
{
return n;
}
else
{
return fibonacci(n - 1) + fibonacci(n - 2);
}
}
// Function to display the Fibonacci series up to n terms
void displayFibonacciSeries(int n)
{
printf("Fibonacci Series up to %d terms:\n", n);
for (int i = 0; i < n; ++i)
{
printf("%d ", fibonacci(i));
}
printf("\n");
}
```

```
int main()
{
int terms;
// Input the number of terms for the Fibonacci series
printf("Enter the number of terms for the Fibonacci series: ");
scanf("%d", &terms);
// Display the Fibonacci series
displayFibonacciSeries(terms);
return 0;
}
```

10.(ii) WRITE A RECURSIVE FUNCTION TO FIND LCM OF TWO NUMBER

SOURCE CODE:

```
#include <stdio.h>
// Recursive function to calculate the least common multiple (LCM)
int lcm_recursive(int a, int b, int current_multiple){
if (current_multiple % a == 0 && current_multiple % b == 0){
return current_multiple;
}
else{
return lcm_recursive(a, b, current_multiple + 1);
}
}
int main() {
int num1, num2;
// Input two numbers
printf("Enter the first number: ");
scanf("%d", &num1);
printf("Enter the second number: ");
scanf("%d", &num2);
// Calculate and display the LCM
int result = lcm_recursive(num1, num2, (num1 > num2) ? num1 : num2);
printf("LCM of %d and %d is: %d\n", num1, num2, result);
return 0;
}
```

10.(iii) WRITE A RECURSIVE FUNCTION TO FIND THE FACTORIAL OF A NUMBER

SOURCE CODE:

```
#include<stdio.h>
int fact(int);
int main()
{
int x,n;
printf(" Enter the Number to Find Factorial :");
scanf("%d",&n);
x=fact(n);
printf(" Factorial of %d is %d",n,x);
return 0;
}
int fact(int n)
{
if(n==0)
return(1);
return(n*fact(n-1));
}
```

10.(iv) WRITE A C PROGRAM RO IMPLEMENT ACKERMANN FUNCTION USING RECURSION

SOURCE CODE:

```
#include <stdio.h>
// Recursive implementation of the Ackermann function
int ackermann(int m, int n)
{
if (m == 0)
{
return n + 1;
}
else if (m > 0 && n == 0)
{
return ackermann(m - 1, 1);
}
else
{
return ackermann(m - 1, ackermann(m, n - 1));
}
}
int main()
{
int m, n;
// Input values for m and n
printf("Enter the value of m: ");
scanf("%d", &m);
```

```
printf("Enter the value of n: ");
scanf("%d", &n);
// Calculate and display the result of the Ackermann function
int result = ackermann(m, n);
printf("Ackermann(%d, %d) = %d\n", m, n, result);
return 0;
}
```

10.(v) WRITE A RECURSIVE FUNCTION TO FIND THE SUM OF SERIES

SOURCE CODE:

```
#include <stdio.h>
#include <math.h>
#define ll long long int
// Recursive function to return
// the sum of the given series
ll sum(int n){
// 1^1 = 1
if (n == 1)
return 1;
else
// Recursive call
return ((ll)pow(n, n) + sum(n - 1));
}
int main()
{
int n;
// Input the value of n from the user
printf("Enter the value of n: ");
scanf("%d", &n);
// Calculate and print the sum of the series
printf("Sum of the series: %lld\n", sum(n));
return 0;
}
```

11.(i) WRITE A C PROGRAM TO SWAP TWO NUMBERS USING CALL BY REFERENCE

SOURCE CODE:

```
#include <stdio.h>
// Function to swap two numbers using call by reference
void swap(int *a, int *b){
int temp = *a;
*a = *b;
*b = temp;
}
int main(){
int num1, num2;
printf("Enter the first number: ");
scanf("%d", &num1);
printf("Enter the second number: ");
scanf("%d", &num2);
printf("\nBefore swapping:\n");
printf("First number: %d\n", num1);
printf("Second number: %d\n", num2);
// Call the swap function
swap(&num1, &num2);
printf("\nAfter swapping:\n");
printf("First number: %d\n", num1);
printf("Second number: %d\n", num2);
return 0;
}
```

11.(ii) DEMONSTRATE DANGLING POINTER PROBLEM USING A C PROGRAM

SOURCE CODE:

```
#include <stdio.h>
#include <stdlib.h>
int* createAndReturnInt()
{
int num = 42;
int *ptr = (int*)malloc(sizeof(int));
if (ptr == NULL)
{
fprintf(stderr, "Memory allocation failed\n");
exit(EXIT_FAILURE);
}
*ptr = num;
return ptr;
}
int main()
{
int *danglingPointer;
// Creating a dangling pointer
danglingPointer = createAndReturnInt();
// Accessing the value through the dangling pointer
printf("Dangling Pointer Value: %d\n", *danglingPointer);
// Deallocating the memory (simulating a bug where the memory is freed)
free(danglingPointer);
```

```
// Accessing the value through the dangling pointer after deallocation
// This leads to undefined behavior (dangling pointer)
printf("Dangling Pointer Value After Deallocation: %d\n", *danglingPointer);
return 0;
}
```

11.(iii) WRITE A C PROGRAM TO COPY ONE STRING INTO ANOTHER USING PROGRAM

SOURCE CODE:

```
#include <stdio.h>
#include <string.h>
// Function to copy one string to another using pointers
void copyStrings(char *source, char *destination){
while (*source){
*destination = *source;
source++;
destination++;
}
*destination = '\0'; // Terminate the destination string
}
int main(){
char source[100], destination[100];
// Input the source string
printf("Enter the source string:\n");
fgets(source, sizeof(source), stdin);
source[strcspn(source, "\n")] = '\0'; // Remove newline character if present
// Copy the source string to destination
copyStrings(source, destination);
// Display the copied string
printf("Copied string is: %s\n", destination);
return 0;
}
```

11.(iv) WRITE A C PROGRAM TO FIND NO OF LOWERCASE, UPPERCASE, DIGITS AND OTHERCHARACTERS USING POINTERS

SOURCE CODE:

```
#include <stdio.h>

void findCharacterTypes(const char *str, int *lowercase, int *uppercase, int *digits,
int *others)

{

while (*str)

{

if (*str >= 'a' && *str <= 'z')

{

(*lowercase)++;

} else if (*str >= 'A' && *str <= 'Z')

{

(*uppercase)++;

}

else if (*str >= '0' && *str <= '9')

{

(*digits)++;

}

else

{

(*others)++;

}

str++;
```

```
}
}
int main()
{
char inputString[100];
// Input the string
printf("Enter a string:\n");
fgets(inputString, sizeof(inputString), stdin);
int lowercase = 0, uppercase = 0, digits = 0, others = 0;
// Find character types using pointers
findCharacterTypes(inputString, &lowercase, &uppercase, &digits, &others);
// Display the results
printf("Number of lowercase characters: %d\n", lowercase);
printf("Number of uppercase characters: %d\n", uppercase);
printf("Number of digits: %d\n", digits);
printf("Number of other characters: %d\n", others);
return 0;
}
```

12.(i) WRITE A C PROGRAM TO FIND THE SUM OF 1D ARRAY USING MALLOC( )

SOURCE CODE:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
int n, *arr, sum = 0;
printf("Enter the size of the array: ");
scanf("%d", &n);
arr = (int *)malloc(n * sizeof(int));
if (arr == NULL) {
printf("Memory allocation failed.\n");
return 1;
}
printf("Enter the elements of the array:\n");
for (int i = 0; i < n; ++i) {
scanf("%d", &arr[i]);
sum += arr[i];
}
printf("Array elements: ");
for (int i = 0; i < n; ++i) printf("%d ", arr[i]);
printf("\nSum of the array elements: %d\n", sum);
return 0;
}
```

12.(ii) WRITE A C PROGRAM TO FIND THE TOTAL, AVERAGE OF N STUDENTS USING STRUCTURES

SOURCE CODE:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_MARKS 100
// Define a structure to store student details
struct Student
{
char name[50];
int *marks; // Dynamic array to store marks for a variable number of subjects
int total;
float average;
};
int main()
{
int n, numSubjects;
// Input the number of students
printf("Enter the number of students: ");
scanf("%d", &n);
// Input the number of subjects with validation
do
{
printf("Enter the number of subjects: ");
scanf("%d", &numSubjects);
if (numSubjects <= 0)
```

```
{
printf("Invalid number of subjects. Please enter a positive number.\n");
}
}
while (numSubjects <= 0);
// Declare an array of structures to store information for 'n' students
struct Student students[n];
// Input details for each student
for (int i = 0; i < n; ++i)
{
printf("Enter details for student %d:\n", i + 1);
printf("Name: ");
scanf("%s", students[i].name);
// Allocate memory for marks based on the number of subjects
students[i].marks = (int *)malloc(numSubjects * sizeof(int));
if (students[i].marks == NULL)
{
printf("Memory allocation failed.\n");
return 1;
}
students[i].total = 0; // Initialize total to 0 for each student
// Input marks for each subject with validation
for (int j = 0; j < numSubjects; ++j)
{
do
{
printf("Enter marks for subject %d (0-%d): ", j + 1, MAX_MARKS);
```

```
scanf("%d", &students[i].marks[j]);
if (students[i].marks[j] < 0 || students[i].marks[j] > MAX_MARKS) {
printf("Invalid marks. Please enter a number between 0 and %d.\n", MAX_MARKS);
}
}
while (students[i].marks[j] < 0 || students[i].marks[j] > MAX_MARKS);
// Calculate total for each student
students[i].total += students[i].marks[j];
}
// Calculate average for each student
students[i].average = (float)students[i].total / numSubjects;
}
// Display the results for each student
printf("\nStudent Details:\n");
for (int i = 0; i < n; ++i) {
printf("Student %d\n", i + 1);
printf("Name: %s\n", students[i].name);
printf("Total Marks: %d\n", students[i].total);
printf("Average Marks: %.2f\n", students[i].average);
printf("\n");
// Free memory allocated for marks
free(students[i].marks);
}
return 0;
}
```

12.(iii) ENTER N STUDENTS DATA USING CALLOC( ) AND DISPLAY FAILED STUDENTS LIST

SOURCE CODE:

```
#include <stdio.h>
#include <stdlib.h>
#define PASSING_MARKS 40
#define MAX_MARKS 100
struct Student
{
char name[50];
int *marks;
int total;
float average;
int failed; // Added a flag to track failed status
};
int main()
{
int n, numSubjects;
printf("Enter the number of students: ");
scanf("%d", &n);
printf("Enter the number of subjects: ");
scanf("%d", &numSubjects);
struct Student *students = (struct Student *)calloc(n, sizeof(struct Student));
if (!students)
{
printf("Memory allocation failed.\n");
```

```
return 1;
}
for (int i = 0; i < n; ++i)
{
printf("\nEnter details for student %d:\nName: ", i + 1);
scanf("%s", students[i].name);
students[i].marks = (int *)calloc(numSubjects, sizeof(int));
students[i].total = 0;
students[i].failed = 0;
for (int j = 0; j < numSubjects; ++j)
{
do
{
printf("Enter marks for subject %d (0-%d): ", j + 1, MAX_MARKS);
scanf("%d", &students[i].marks[j]);
}
while (students[i].marks[j] < 0 || students[i].marks[j] > MAX_MARKS);
students[i].total += students[i].marks[j];
if (students[i].marks[j] < PASSING_MARKS)
{
students[i].failed = 1;
}
}
students[i].average = (float)students[i].total / numSubjects;
}
printf("\nPass and Fail Information:\n");
for (int i = 0; i < n; ++i)
```

```
{
printf("Student %d: %s - %s\n", i + 1, students[i].name, (students[i].failed ? "Failed" :
"Passed"));
}
return 0;
}
```

12.(iv) READ STUDENT NAME AND MARKS FROM COMMAND LINE AND DISPLAY THE STUDENT DETAILS ALONG WITH THE TOTAL

SOURCE CODE:

```
#include <stdio.h>

#include <stdlib.h>

int main(int argc, char *argv[])

{

// Check if the correct number of command line arguments is provided

if (argc < 7)

{

printf("Usage: %s <student_name> <marks_subject1> <marks_subject2>
<marks_subject3>

<marks_subject4> <marks_subject5>\n", argv[0]);

return 1; // Exit with an error code

}

// Parse the command line arguments

char *studentName = argv[1];

float marks_subject1 = atof(argv[2]); // Convert marks from string to float

float marks_subject2 = atof(argv[3]); // Convert marks from string to float

float marks_subject3 = atof(argv[4]); // Convert marks from string to float

float marks_subject4 = atof(argv[5]); // Convert marks from string to float

float marks_subject5 = atof(argv[6]); // Convert marks from string to float

// Check if marks are valid

if (marks_subject1 < 0 || marks_subject1 > 100 ||

marks_subject2 < 0 || marks_subject2 > 100 ||
```

```
marks_subject3 < 0 || marks_subject3 > 100 ||
marks_subject4 < 0 || marks_subject4 > 100 ||
marks_subject5 < 0 || marks_subject5 > 100)
{
printf("Invalid marks. Marks should be between 0 and 100.\n");
return 1; // Exit with an error code
}
// Calculate total for all subjects
float total = marks_subject1 + marks_subject2 + marks_subject3 + marks_subject4 +
marks_subject5;
// Display student details along with the total
printf("Student Name: %s\n", studentName);
printf("Marks Subject 1: %.2f\n", marks_subject1);
printf("Marks Subject 2: %.2f\n", marks_subject2);
printf("Marks Subject 3: %.2f\n", marks_subject3);
printf("Marks Subject 4: %.2f\n", marks_subject4);
printf("Marks Subject 5: %.2f\n", marks_subject5);
printf("Total: %.2f\n", total);
return 0; // Exit successfully
}
```

12.(v) WRITE A C PROGRAM TO IMPLEMENT REALLOC( )

SOURCE CODE:

```
#include <stdio.h>
#include <stdlib.h>
void* my_realloc(void* ptr, size_t size)
{
if (size == 0)
{
free(ptr);
return NULL;
}
if (ptr == NULL)
{
return malloc(size);
}
else
{
void* new_ptr = malloc(size);
if (new_ptr != NULL)
{
size_t old_size = malloc_usable_size(ptr);
size_t copy_size = (old_size < size) ? old_size : size;
memcpy(new_ptr, ptr, copy_size);
free(ptr);
}
```

```
return new_ptr;
}
}
int main()
{
// Example usage
int *arr = (int*)malloc(3 * sizeof(int));
if (arr == NULL)
{
fprintf(stderr, "Memory allocation failed.\n");
return 1;
}
// Initialize the array
for (int i = 0; i < 3; i++)
{
arr[i] = i;
}
// Print the original array
printf("Original array: ");
for (int i = 0; i < 3; i++)
{
printf("%d ", arr[i]);
}
printf("\n");
// Resize the array using my_realloc
arr = my_realloc(arr, 5 * sizeof(int));
// Check if realloc was successful
```

```
if (arr == NULL)
{
fprintf(stderr, "Memory reallocation failed.\n");
return 1;
}
// Initialize the new elements in the resized array
for (int i = 3; i < 5; i++)
{
arr[i] = i;
}
// Print the resized array
printf("Resized array: ");
for (int i = 0; i < 5; i++)
{
printf("%d ", arr[i]);
}
printf("\n");
// Free the allocated memory
free(arr);
return 0;
}
```

13.(i) READ AND PRINT A DATE USING DD/MM/YYYY FORMAT USING BIT-FIELDS AND DIFFERENTIATE THE SAMEWITHOUT USING BIT-FIELDS

SOURCE CODE:

```
#include <stdio.h>
// Define a structure for date using bit-fields
struct DateWithBitFields
{
unsigned int day : 5, month : 2, year : 12;
};
// Define a structure for date without using bit-fields
struct DateWithoutBitFields
{
unsigned int day, month, year;
};
// Function to print date in dd/mm/yyyy format using bit-fields
void printDateWithBitFields(struct DateWithBitFields date)
{
printf("%02u/%02u/%04u\n", date.day, date.month, date.year);
}
// Function to print date in dd/mm/yyyy format without using bit-fields
void printDateWithoutBitFields(struct DateWithoutBitFields date)
{
printf("%02u/%02u/%04u\n", date.day, date.month, date.year);
}
int main()
```

```
{
// Example with bit-fields
struct DateWithBitFields dateWithBitFields;
unsigned int tempDay, tempMonth, tempYear;
printf("Enter date with bit-fields (dd mm yyyy): ");
scanf("%u %u %u", &tempDay, &tempMonth, &tempYear);
dateWithBitFields.day = tempDay;
dateWithBitFields.month = tempMonth;
dateWithBitFields.year = tempYear;
printf("Date with bit-fields: ");
printDateWithBitFields(dateWithBitFields);
// Example without bit-fields
struct DateWithoutBitFields dateWithoutBitFields;
printf("\nEnter date without bit-fields (dd mm yyyy): ");
scanf("%u %u %u", &dateWithoutBitFields.day, &dateWithoutBitFields.month,
&dateWithoutBitFields.year);
printf("Date without bit-fields: ");
printDateWithoutBitFields(dateWithoutBitFields);
return 0;
}
```

13.(ii) CREATE AND DISPLAY A SINGLY LIST USING SELF-REFERENTIAL STRUCTURE

SOURCE CODE:

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
int data;
struct Node *next;
};
struct Node* createNode(int value)
{
struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
if (!newNode)
{
perror("Memory allocation failed");
exit(EXIT_FAILURE);
}
newNode->data = value;
newNode->next = NULL;
return newNode;
}
void insertAtEnd(struct Node** head, int value)
{
struct Node* newNode = createNode(value);
if (!*head)
```

```
{
*head = newNode;
}
else
{
struct Node* current = *head;
while (current->next) {
current = current->next;
}
current->next = newNode;
}
}
void displayList(struct Node* head)
{
printf("Linked List: ");
while (head)
{
printf("%d ", head->data);
head = head->next;
}
printf("\n");
}
void freeList(struct Node* head)
{
while (head)
{
struct Node* temp = head;
```

```
head = head->next;
free(temp);
}
}
int main()
{
struct Node* head = NULL;
char choice;
do
{
int value;
printf("Enter a value for the linked list: ");
scanf("%d", &value);
insertAtEnd(&head, value);
printf("Do you want to enter another value? (y/n): ");
scanf(" %c", &choice);
}
while (choice == 'y' || choice == 'Y');
displayList(head);
freeList(head);
return 0;
}
```

13.(iii) DEMONSTRATE THE DIFFERENCE BETWEEN STRUCTURE AND UNIONS USING A C PROGRAM

SOURCE CODE:

```
#include <stdio.h>
// Define a structure
struct Point
{
int x;
int y;
};
// Define a union
union Coordinate
{
int x;
int y;
};
int main()
{
// Structure example
struct Point pointStruct;
pointStruct.x = 10;
pointStruct.y = 20;
printf("Structure Example:\n");
printf("X-coordinate: %d\n", pointStruct.x);
printf("Y-coordinate: %d\n", pointStruct.y);
printf("Size of Structure: %lu bytes\n", sizeof(pointStruct));
```

```
printf("\n");
// Union example
union Coordinate coordinateUnion;
coordinateUnion.x = 30;
printf("Union Example:\n");
printf("X-coordinate: %d\n", coordinateUnion.x);
// Accessing y-coordinate after setting x-coordinate
printf("Y-coordinate: %d\n", coordinateUnion.y);
printf("Size of Union: %lu bytes\n", sizeof(coordinateUnion));
return 0;
}
```

13.(iv) WRITE A C PROGRAM TO SHIFT/ROTATE USING BITFIELDS

SOURCE CODE:

```
#include <stdio.h>
// Define a structure with a 4-bit bitfield
struct BitFieldExample
{
unsigned int value : 4;
};
// Function to perform left circular shift on the bitfield
void leftCircularShift(struct BitFieldExample *bf)
{
bf->value = (bf->value << 1) | (bf->value >> 3);
}
// Function to perform right circular shift on the bitfield
void rightCircularShift(struct BitFieldExample *bf)
{
bf->value = (bf->value >> 1) | (bf->value << 3);
}
int main()
{
struct BitFieldExample bf;
unsigned int inputValue;
// Get input from the user
printf("Enter a 4-bit value in hexadecimal (e.g., 0x3): ");
scanf("%x", &inputValue);
```

```
// Ensure only 4 bits are considered
bf.value = inputValue & 0xF;
// Display the initial value
printf("Initial Value: 0x%X\n", bf.value);
// Perform left circular shift
leftCircularShift(&bf);
printf("After Left Circular Shift: 0x%X\n", bf.value);
// Perform right circular shift
rightCircularShift(&bf);
printf("After Right Circular Shift: 0x%X\n", bf.value);
return 0;
}
```

13(v) WRITE A C PROGRAM TO COPY ONE STRUCTURE VARIABLE TO ANOTHER STRUCTURE OF THE SAME TYPE

SOURCE CODE:

```
#include <stdio.h>
#include <string.h>
// Define the structure
struct Person
{
char name[50];
int age;
};
// Function to copy one structure to another
void copyPerson(struct Person *dest, const struct Person *src)
{
// Use the assignment operator to copy individual members
strcpy(dest->name, src->name);
dest->age = src->age;
}
int main()
{
// Declare two structure variables
struct Person person1, person2;
// Get input for Person 1
printf("Enter details for Person 1:\n");
printf("Name: ");
scanf("%s", person1.name); // Assuming that the name doesn't contain spaces
```

```
printf("Age: ");

scanf("%d", &person1.age);

// Copy the contents of person1 to person2

copyPerson(&person2, &person1);

// Display the contents of both structures

printf("\nPerson 1:\nName: %s\nAge: %d\n\n", person1.name, person1.age);

printf("Person 2 (copied from Person 1):\nName: %s\nAge: %d\n", person2.name,
person2.age);

return 0;

}
```

14.(i) WRITE A C PROGRAM TO WRITE AND READ TEXT INTO A FILE

SOURCE CODE:

```
#include <stdio.h>
int main() {
FILE *file;
char fileName[] = "example.txt";
char buffer[100];
char textToWrite[100];
// Get text from the user
printf("Enter text to write to the file: ");
fgets(textToWrite, sizeof(textToWrite), stdin);
// Writing text to a file
file = fopen(fileName, "w");
fprintf(file, "%s", textToWrite);
fclose(file);
printf("Text written to the file.\n");
// Reading text from the file
file = fopen(fileName, "r");
printf("Reading text from the file:\n");
while (fgets(buffer, sizeof(buffer), file) != NULL) {
printf("%s", buffer);
}
fclose(file);
return 0;
}
```

14.(ii) WRITE A C PROGRAM TO WRITE AND READ TEXT INTO A BINARY FILE USING FREAD( ) AND FWRITE( )

SOURCE CODE:

```
#include <stdio.h>
int main()
{
FILE *file;
char fileName[] = "binary.dat";
char buffer[100];
char textToWrite[100];
// Get text from the user
printf("Enter text to write to the binary file: ");
fgets(textToWrite, sizeof(textToWrite), stdin);
// Writing text to a binary file
if ((file = fopen(fileName, "wb")) == NULL)
{
perror("Error opening file for writing");
return 1;
}
fwrite(textToWrite, sizeof(char), sizeof(textToWrite), file);
fclose(file);
printf("Text written to the binary file.\n");
// Reading text from the binary file
if ((file = fopen(fileName, "rb")) == NULL)
{
perror("Error opening file for reading");
```

```
return 1;
}
printf("Reading text from the binary file:\n");
fread(buffer, sizeof(char), sizeof(textToWrite), file);
printf("%s", buffer);
fclose(file);
return 0;
}
```

14.(iii) COPY THE CONTENTS OF ONE FILE TO ANOTHER FILE

SOURCE CODE:

```
#include <stdio.h>
int main()
{
FILE *sourceFile, *destinationFile;
char sourceFileName[50], destinationFileName[50];
char buffer[100];
// Get source and destination filenames from the user
printf("Enter source file name: ");
scanf("%s", sourceFileName);
// Open the source file for reading
if ((sourceFile = fopen(sourceFileName, "r")) == NULL)
{
perror("Error opening source file for reading");
return 1;
}
printf("Enter destination file name: ");
scanf("%s", destinationFileName);
// Open the destination file for writing
if ((destinationFile = fopen(destinationFileName, "w")) == NULL)
{
perror("Error opening destination file for writing");
fclose(sourceFile);
return 1;
```

```
}
// Copy content from source file to destination file
while (fgets(buffer, sizeof(buffer), sourceFile) != NULL)
{
fputs(buffer, destinationFile);
}
printf("Contents copied from %s to %s.\n", sourceFileName, destinationFileName);
// Close both files
fclose(sourceFile);
fclose(destinationFile);
return 0;
}
```

14.(iv) WRITE A C PROGRAM TO MERGE TWO FILES INTO THE THIRD FILE USING COMMAND LINE ARGUMENTS

SOURCE CODE:

```
#include <stdio.h>
int main() {
FILE *file1, *file2, *mergedFile;
char buffer[100];
char file1Name[50], file2Name[50], mergedFileName[50];
// Get file names from the user
printf("Enter the name of the first file: ");
scanf("%s", file1Name);
// Open the first file for reading
if ((file1 = fopen(file1Name, "r")) == NULL)
{
perror("Error opening the first file for reading");
return 1;
}
printf("Enter the name of the second file: ");
scanf("%s", file2Name);
// Open the second file for reading
if ((file2 = fopen(file2Name, "r")) == NULL) {
perror("Error opening the second file for reading");
fclose(file1);
return 1;
}
printf("Enter the name of the merged file: ");
```

```
scanf("%s", mergedFileName);
// Open the merged file for writing
if ((mergedFile = fopen(mergedFileName, "w")) == NULL) {
perror("Error opening the merged file for writing");
fclose(file1);
fclose(file2);
return 1;
}
// Merge content from the first file to the merged file
while (fgets(buffer, sizeof(buffer), file1) != NULL)
{
fputs(buffer, mergedFile);
}
// Merge content from the second file to the merged file
while (fgets(buffer, sizeof(buffer), file2) != NULL)
{
fputs(buffer, mergedFile);
}
printf("Contents of %s and %s merged into %s.\n", file1Name, file2Name,
mergedFileName);
// Close all files
fclose(file1);
fclose(file2);
fclose(mergedFile);
return 0;
}
```

14.(v) FIND NO.OF LINES, WORDS AND CHARACTERS IN A FILE

SOURCE CODE:

```
#include <stdio.h>
int main()
{
FILE *file;
char fileName[50];
char ch;
int lines = 0, words = 0, characters = 0, inWord = 0;
// Get the file name from the user
printf("Enter the name of the file: ");
scanf("%s", fileName);
// Open the file for reading
if ((file = fopen(fileName, "r")) == NULL)
{
perror("Error opening file for reading");
return 1;
}
// Count lines, words, and characters in the file
while ((ch = fgetc(file)) != EOF)
{
characters++;
if (ch == '\n')
{
lines++;
```

```
}
// Check for word boundaries
if (ch == ' ' || ch == '\t' || ch == '\n')
{
inWord = 0;
}
else if (inWord == 0)
{
inWord = 1;
words++;
}
}
printf("Number of lines: %d\n", lines);
printf("Number of words: %d\n", words);
printf("Number of characters: %d\n", characters);
// Close the file
fclose(file);
return 0;
}
```

14.(vi) WRITE A C PROGRAM TO PRINT LAST N CHARACTERS OF A GIVEN LIST

SOURCE CODE:

```
#include <stdio.h>
#include <stdlib.h>
void printLastNCharacters(FILE *file, int n)
{
fseek(file, 0, SEEK_END); // Move the file pointer to the end of the file
long fileSize = ftell(file); // Get the total size of the file
// Read and print the last 'n' characters
for (long i = 1; i <= n && i <= fileSize; i++)
{
fseek(file, -i, SEEK_END);
char ch = fgetc(file);
putchar(ch);
}
}
int main()
{
FILE *file;
char fileName[50];
int n;
// Get the file name from the user
printf("Enter the name of the file: ");
scanf("%s", fileName);
// Open the file for reading
```

```
if ((file = fopen(fileName, "r")) == NULL)
{
perror("Error opening file for reading");
return 1;
}
// Get the value of 'n' from the user
printf("Enter the value of 'n': ");
scanf("%d", &n);
printf("Last %d characters of the file:\n", n);
printLastNCharacters(file, n);
// Close the file
fclose(file);
return 0;
}
```